Carleton University

QNX Quantum Secure OTA Update

COMP4900E, Real Time Operating Systems

Final Project Report

Ben Whitten BenWhitten@cmail.carleton.ca
Jacob Bonner JacobBonner@cmail.carleton.ca
Caitlin Wardle CaitlinWardle@cmail.carleton.ca

April 7, 2026

Abstract

This project investigates whether a post-quantum digital signature scheme can be integrated into a QNX over-the-air (OTA) update workflow without making the update process impractical for embedded devices. Our target platform is a Raspberry Pi 4 running QNX OS 8.0, and our core comparison is between three update modes, unsigned OTA, classically signed OTA using RSA and Post Quantum Computing OTA using ML-DSA. The system is designed around an A/B partition update model in which a device checks a remote server for a new image, downloads an update bundle, verifies its signature, writes the new image to the inactive partition and reboots into the updated system if the validation succeeds. The report presents the need for secure OTA, the quantum risks facing current signing techniques, the benefits of ML-DSA, the implementation architecture and the comparisons across algorithms. Our evaluation focuses on signature generation time, signature verification time, total OTA update time, CPU usage, RAM usage, signature size, and storage overhead.

Contents

1 Introduction

Embedded systems in automotive, medical, industrial and IoT environments usually rely on remote software updates to remain secure and functional over their deployment lifespan. Over the air (OTA) updates allow new software images or configuration changes to be delivered over a network, so physical access of the device is not required. OTA typically involves packaging an update on a server, transmitting it to a device, verifying its integrity and authenticity on the device, installing it and finally rebooting into the new version.[1] For embedded systems OTA is beneficial because it reduces maintenance cost, minimizes system downtime, allows for rapid deployment and satisfies consumer need for at home maintenance. However, OTA has one critical problem, if an attacker can forger or tamper with an update, then they can gain persistent control of the device, a potentially disastrous scenario.[2] This risk is exacerbated by the long lifespans of many embedded devices, since cryptographic assumptions used today may no longer be secure over the full life of the product. Most OTA systems rely on classical public key encryption schemes, like RSA or elliptic-curve cryptography. But the long term security of these algorithms is threatened by large scale quantum computing, since new methods, like Shor's algorithm would break the hard mathematical problems these encryption schemes currently depend on.[3] As such it is important to investigate whether post quantum signature mechanisms can be integrated into OTA validation, without unacceptable cost to the devices performance. This project aims to address that question by designing and evaluating a secure OTA mechanism for QNX based embedded systems using a Post Quantum Computing (PQC) signature algorithm for update validation. Our implementation is for QNX OS 8.0 on a Rasberry Pi 4, and uses ML-DSA through liboqs as the pqc signature scheme, with RSA through OpenSSL as the classical comparison baseline. As a baseline we have also included unsigned OTA, to isolate the cost of signature validation from the cost of the update process.

1.1 Research Question

Is it feasible to replace a classical code signing algorithm in a QNX OTA update with a post quantum computing signature scheme without causing significant performance degradation on a simple embedded device?

1. Siara Singleton, "OTA for IoT: What It Is, How It Works, and Why It Matters," Memfault, April 1, 2025, accessed April 7, 2026, https://memfault.com/blog/ota-for-iot/.

2. Alexander Truskovsky, "Do You Know About Quantum's Threat to OTA Software Updates?," ISARA, September 17, 2017, accessed April 7, 2026, https://www.isara.com/blog-posts/quantums-threat-ota-software-updates.html.

3. "Understanding Shor's And Grover's Algorithms And Their Impact On Cybersecurity," Fortinet, 2025, accessed April 7, 2026, https://www.fortinet.com/resources/cyberglossary/shors-grovers-algorithms.

1.2 Objectives

1. Implement a working OTA update prototype on QNX 8.0 for Raspberry Pi 4.
2. Integrate a digital signature verification step into the OTA validation path.
3. Implement RSA signing and verification as a baseline.
4. Implement ML-DSA signing and verification using liboqs as the post quantum solution
5. Compare unsigned, RSA signed and ML-DSA signed updates using timing and resource usage.
6. Analyze the resource demands of post quantum computing signatures on OTA updates in embedded devices to determine a baseline of resource and time requirements. As individual embedded systems have specific requirements the praticality of these drains will need to be determined on a per system basis.

1.3 Scope

The scope of this project is the OTA image packaging, download, signature validation, installation to an inactive partition and the reboot on an embedded device. The focus is on code signing and verification, not on transport encryption, key exchange or large scale deployments. Within this scope the project has three parts. Firstly it builds an OTA pipeline for QNX on Raspberry Pi 4 using an A/B partition model. Secondly it builds both a classical (RSA) and a post quantum computing (ML-DSA) signature into the update validation mechanism. Finally, it collects data pursuant to algorithm cost and device level measurements like runtime, CPU load, memory consumption and storage overhead.

1.4 Contributions

- **Ben Whitten:**
  - SHA256 hash code.
  - RSA signing and verification code.
  - MLDSA signing code.
  - Troubleshooting.
  - Testing and analysis.
  - Project Proposal: Study method.
  - Final Report: Sections 4.6, 4.7, 6-8.

- **Jacob Bonner:**
  - Boot Process.
  - Image Creation/Configuration.
  - Raspberry Pi Configuration.
  - Troubleshooting.
  - Project Proposal: Development Technologies.
  - Final Report: Sections 5, 7.2, 7.3.
- **Caitlin Wardle:**
  - ML-DSA Verification.
  - OTA Code.
  - Data Collection Code.
  - Troubleshooting.
  - Project Proposal: Domain Description, Research Question.
  - Final Report: Sections 1, 2, 3, 4.1 - 4.5, 7.2.

2 Background

2.1 OTA Updates in Embedded Systems

OTA updates allow a device to receive a new software image over a network connection rather than through physical access. A typical OTA pipeline packages an image on a server, transfers the image to the embedded device, validates the update on the device, installs the update, and reboots into the new version. For embedded systems, OTA is especially useful for remote or critical devices.[4] Despite the benefits, OTA updates are more vulnerable to installation failures and malicious attacks. If a device loses power, writes an incomplete image, or boots an invalid update, it may become unusable or "bricked."[5] For this reason, OTA systems rely on rollback capabilities,saved versions, and cryptographic verification. A secure OTA design must protect against accidental failure and against malicious changes.

4. Singleton, "OTA for IoT: What It Is, How It Works, and Why It Matters."

5. Siara Singleton, "How to Test OTA Updates Without Bricking Devices," Memfault, May 15, 2025, accessed April 7, 2026, https://memfault.com/blog/ota-testing-101-the-ultimate-guide.

2.2 Why Quantum Resistance Matters

The signature verification step in current OTA systems uses classic public key cryptography. The device stores a trusted public key and uses it to verify that the received update was signed by the trusted private key holder. This model works well today, but its long term security depends on mathematical formulas that future quantum computers may be able to break. Shor's algorithm would make integer factorization and elliptic curve discrete logarithms solvable, destroying the security of RSA and ECC.[6] Since many embedded systems have long life spans, a device currently in use may still rely on these algorithms long after quantum attacks become effective. The *harvest now, forge later* threat model proposes attackers can collect update data and system images now and break them in the future, when quantum computers are developed.[7]

2.3 Post Quantum Signatures

Post quantum computing (PQC) refers to cryptographic algorithms designed to remain secure even against attackers with quantum computers. For digital signatures the major categories of PQC are lattice based, hash based and multivariate constructions.[8] This project focuses on ML-DSA, a lattice based digital signature algorithm standardized by NIST and supported by liboqs.[9] ML-DSA was chosen because it offers a strong balance between security and practical performance for constrained systems. Compared to other algorithms like SLH-DSA, ML-DSA generally provides smaller signatures and faster signing and verification, which are important when working with embedded devices operating with limited resources.[10]

2.4 Classical Signatures

To determine the baseline overhead of current validation mechanisms this project has also implemented a classical scheme (RSA) that reflects present day OTA validations. The project uses RSA via OpenSSL as our baseline baseline classical signature mechanism. RSA is well understood, widely supported, and easy to implement in a comparable signing and

6. "Understanding Shor's And Grover's Algorithms And Their Impact On Cybersecurity."
7. Marin Ivezic, "Trust Now, Forge Later (TNFL) – The Overlooked Quantum Threat," ISARA, September 10, 2025, accessed April 7, 2026, https://postquantum.com/post-quantum/trust-now-forge-later/.
8. "Post-Quantum Cryptography," NIST, December 11, 2025, accessed April 7, 2026, https://www.nist.gov/pqcrypto.
9. "Module-Lattice-Based Digital Signature Standard," NIST, August 13, 2024, accessed April 7, 2026, https://csrc.nist.gov/pubs/fips/204/final.
10. "ML-DSA A Quantum Cryptography Solution for Digital Signing," digicert, April 7, 2024, accessed April 7, 2026, https://www.digicert.com/insights/post-quantum-cryptography/mldsa.

verification workflow.[11]

3 System Design

3.1 Hardware

The implementation was preformed with a Rasberry PI 4 running QNX OS 8.0. The Raspberry Pi 4 was chosen because it is affordable, well documented and practical for testing simple embedded devices. Using real hardware allows for device level measurements to realistically capture the performance limits and operational constrains of actual embedded systems.[12]

3.2 A/B Partition OTA Model

The OTA system is designed around an A/B partition architecture. In this model, partition A contains the currently running system image, while partition B is inactive and can be used as the destination for a downloaded update. Firstly the device will check the remote server for a new image. If a new image is available, the device downloads the update package. Then the device verifies the digital signature associated with the update. If the verification is successful, the new image is written to inactive partition B. The boot configuration is changed so the device boots from the updated partition B. The system reboots into the new image. This design has excellent rollback capabilities, if the verification fails, installation is aborted. If the update is written but becomes unstable, the system can fall back to the previously working partition.[13] This makes the A/B model robust, and ideal for OTA.

3.3 Threat Model

The assumed advisory for this project is able to observe the update channel, replaying old update bundles, or replacing legitimate images or signatures with malicious ones during transfer or staging. The main security consideration is authenticity and integrity for the OTA image before installation. That is why the device recomputes the image digest locally and verifies a signature under a trusted public key before switching boot targets. Notably, this project does not attempt to solve every problem within the secure software delivery

11. "RSA Algorithm in Cryptography," geeksforgeek, July 23, 2025, accessed April 7, 2026, https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/.
12. "Raspberry Pi computer hardware," Rasberry Pi, accessed April 7, 2026, https://www.raspberrypi.com/documentation/computers/raspberry-pi.html.
13. Josef Holzmayr, "Robust OTA updates with A/B Partitions for Linux devices," Mender, March 26, 2025, accessed April 7, 2026, https://mender.io/blog/robust-ota-updates-with-partitions-for-linux-devices.

pipeline. It does not address compromise of the build server, theft of the signing private key, physical attacks on the device or confidentiality of the updates contents. While these concerns are important in OTA systems they are unrelated to the narrow question this report investigates; whether replacing a classical signature scheme with a post quantum one is operationally feasible in a QNX OTA path.

3.4 Cryptographic Workflow

The Cryptographic workflow consists of two sides, the server side packaging step and the device side verification step.

Server Side:

1. Build or package the QNX image using `mkqnximage`[14]
2. Compute a SHA 256 digest of the image
3. Sign the digest using RSA or ML-DSA
4. Publish the update image, signature and metadata to the server

Device Side:

1. Download the update package from server.
2. Recompute the SHA 256 digest of the image.
3. Verify the signature using the stored public key.
4. Abandon installation if verification fails.
5. If verification succeeds, write the image to partition B and reboot.

4 Implementation

4.1 Software

The main technologies used in this project are QNX Momentics IDE, `mkqnximage`, liboqs, OpenSSL, Git/GitHub, and Raspberry Pi 4 hardware. QNX Momentics is used as the main development environment, and the remote server where images are generated and signed.

14. "mkqnximage," QNX, August 11, 2025, accessed April 7, 2026, https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.utilities/topic/m/mkqnximage.html.

`mkqnximage` is used to generate QNX operating system images.[15] OpenSSL provides classical RSA signing and verification,[16] while liboqs provides the ML-DSA implementation for post quantum computing validation.[17] Github acts as storage and version control for the project implementation.

4.2 OTA Update

The unsigned OTA serves as a baseline measurement. In this mode, the device checks for an update, downloads the image, and writes it to the inactive partition without performing any kind of validation. This will establish a minimum time and resource cost of the OTA process itself. Without this baseline this project cannot accurately determine the cost of update retrieval and image download from the cost of the update validation mechanism

4.3 RSA OTA Validation

The RSA method continues the baseline OTA mechanism, but adds a signing step on the server and a verification step on the device. On the server a SHA 256 digest of the update image is signed with an RSA private key through OpenSSL. On the device, the update image is hashed again and the signature is verified with the RSA public key before installation proceeds. This models the classical public key verification currently used in many secure update systems. It provides a comparison point for evaluating the increased cost of post quantum signatures.[18]

4.4 ML-DSA OTA Validation

The post quantum method follows the same logic as RSA but replaces the signature scheme with ML-DSA using liboqs. On the server, the image digest is signed using an ML-DSA private key. On the device, the update digest is recomputed and verified using the corresponding ML-DSA public key. This implementation aims to keep workflow as consistent as possible while only swapping the signature scheme, in order to get a clean comparison of algorithmic cost. The main difference between RSA and ML-DSA is signature size and key size and we suspect this to have a negative impact on verification time.[19]

15. "mkqnximage."
16. "EVP_PKEY-RSA," OpenSSL Documentation, accessed April 7, 2026, https://docs.openssl.org/3.4/man7/EVP_PKEY-RSA/.
17. bhess, "liboqs," GitHub, accessed April 7, 2026, https://github.com/open-quantum-safe/liboqs.
18. "RSA Algorithm in Cryptography."
19. "Module-Lattice-Based Digital Signature Standard," National Institute of Standards and Technology, accessed April 7, 2026, https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.204.pdf.

4.5 Metrics

The following metrics are collected for each update mode:

- **Signature Generation Time**: Time required to create the signature on the server side.
- **Signature Verification Time**: Time required to verify the signature on the device.
- **Total OTA Time**: Full Time required from checking for updates/downloading update through validation to installation.
- **CPU Usage**: CPU cost observed during validation.
- **RAM Usage**: Maximum memory consumed during validation.
- **Signature Size**: Size in bytes of the generated signature.
- **Storage Overhead**: Additional package storage needed for signature components and keys.

4.6 Measurement Procedure

Within the Build file, `ota_update_ifs.sh`, `ota_update_ifs_rsa.sh`, and `ota_update_ifs_mldsa.sh` are responsible for running the three types of OTA update this project supports. They collect local performance and timing data while the update runs. First they start a background loop that calls top every seconds and save the raw output to `ota_top_raw.log`, while creating a csv file for the sampled CPU and RAM usage. After validation and install, the scripts parse those top logs to calculate the max and average CPU and RAM usage, sample count and the total verification time. Finally a summery log is written to `/fs/boot/ota/ota_summary.log`. This data can then be analyzed and collected.

To test the impact of post quantum computing signature algorithms on QNX OTA updates, the update code will be run using each of the decided upon signature schemes: RSA, MLDSA, and unsigned. Each signature scheme will be run 20 times and the metrics listed above in section 4.5 will be collected across each test.

4.7 Data Analysis Procedure

Once the data has been collected according to what's outlined in section 4.6, the mean and standard error of each metric across all tests will be calculated. These results will then be used in sections 6 and 7 to determine whether the adoption of post-quantum cryptographic algorithms is currently feasible or not.

5 OTA Tutorial

The following are the requirements and steps wto replicate our OTA tests.

5.1 Hardware & Software Requirements

Github Repo: https://github.com/BenjaminWhitten/COMP4900-Quantum-Secure-OTA
Host Server PC: Windows 11 (with WSL Ubuntu)
QNX SDP: Version 8.0 (requires BlackBerry QNX license)
QNX Raspberry Pi Board Support: QNX SDP 8.0 BSP for Raspberry Pi BCM2711 R-PI4
Development IDE: QNX Momentics IDE
QNX OS Terminal Interface: PuTTY
Local Server: Windows OpenSSH Server
Micro Controller: Raspberry Pi 4
Bootloader: https://github.com/u-boot/u-boot
Raspberry Pi Firmware: https://github.com/raspberrypi/firmware.git

Required Cables/Accessories:

- USB to TTL serial cable (to access the Pi's UART pins for console)
- Micro SD card (32 GB)
- Ethernet Cable

5.2 Install Necessary Software Tools

The first thing to do will be to make sure all the required software is downloaded.

1) Installing QNX SDP 8.0 and BSP:

1. Make sure you have downloaded the QNX Software Center from the QNX website. To do this, you will first need to create an account apply for a license.
2. Inside the software center, find the QNX SDP 8.0 package and install it Once you do, unzip it.
3. Find the package with the QNX BSP for the Raspberry Pi 4 and install it. Once you do, unzip it.

4. Find the package that contains QNX Momentics IDE and install it. Once you do, make sure you can open the IDE and import directories as projects. Follow any setup steps it prompts.

2) Installing Windows WSL:

1. Open Windows Powershell. Run the command `wsl -install`.
2. Open WSL. Navigate to the directory where you installed QNX SDP 8.0. It should be something similar to `/mnt/c/path-to-qnx-directory/qnx800`.
3. In the qnx800 directory, set up your work environment with this command: `source qnxsdp-env.sh`.

3) Installing Windows OpenSSH Server:

1. Locate OpenSSH Server and install it.
2. Open Windows Powershell. Run the following commands to start the SSH server:
   - `Get-Service ssh-agent, sshd | Set-Service-StartupType Automatic`
   - `Start-Service ssh-agent`
   - `Start-Service sshd`

4) Installing PuTTY:

1. Go to the official PuTTY download page, and download the correct msi file for your system.
2. Locate the msi file and run it to launch the installer. Complete the setup, then make sure you can open the application.

5) Installing Liboqs:

1. Go to the QNX Software Center and make sure you have QNX SDP 8.0 BSP for Rasberry Pi BCM2711 R-P14 downloaded
2. From your local command prompt navigate to you QNX instance and run qnxsdp-env.bat
3. Install CMake and Ninja

```
winget install -e --id Kitware.CMake
winget install -e --id Ninja-build.Ninja
```

4. Build Liboqs

```
cd C:\
git clone -b main https://github.com/open-quantum-safe/liboqs.git
cd liboqs
rmdir /s /q build-qnx-aarch64

cmake -S . -B build-qnx-aarch64 -G Ninja ^
  -DCMAKE_SYSTEM_NAME=QNX ^
  -DCMAKE_SYSTEM_PROCESSOR=aarch64 ^
  -DCMAKE_C_COMPILER=qcc ^
  -DCMAKE_CXX_COMPILER=q++ ^
  -DCMAKE_AR:FILEPATH="C:/Users/harly/qnx800_2/host/win64/x86_64/usr/bi
  -DCMAKE_C_FLAGS="-Vgcc_ntoaarch64le" ^
  -DCMAKE_CXX_FLAGS="-Vgcc_ntoaarch64le" ^
  -DCMAKE_INSTALL_PREFIX=C:/liboqs/build-qnx-aarch64/install ^
  -DOQS_BUILD_ONLY_LIB=ON ^
  -DOQS_MINIMAL_BUILD="SIG_ml_dsa_44" ^
  -DBUILD_SHARED_LIBS=OFF ^
  -DOQS_USE_OPENSSL=OFF ^
  DOQS_DIST_BUILD=OFF ^
  -DOQS_OPT_TARGET=generic ^
  -DOQS_USE_ARM_AES_INSTRUCTIONS=OFF ^
  -DOQS_USE_ARM_SHA2_INSTRUCTIONS=OFF ^
  -DOQS_USE_ARM_SHA3_INSTRUCTIONS=OFF ^
  -DOQS_USE_ARM_NEON_INSTRUCTIONS=OFF

cmake --build build-qnx-aarch64
cmake --install build-qnx-aarch64
```

5.3 Compile Bootloader (U-Boot)

Use WSL and clone the U-Boot Git repository (see section 5.1). Once you have it, follow these steps in WSL:

1. Navigate to the u-boot directory that you cloned.
2. Make sure that you have the necessary libraries and that they are up to date. You can do this with `sudo apt update` followed by `sudo apt install -y git dosfstools gcc-aarch64-linux-gnu make flex bison bc libssl-dev libgnutls28-dev`.
3. Before you build the u-boot binary, create a config with the following command: `CROSS_COMPILE=aarch64-linux-gnu- make rpi_4_defconfig`.
4. Now compile the u-boot binary with this command: `CROSS_COMPILE=aarch64-linux-gnu- make -j$(nproc)`. This should generate a `u-boot.bin` file.[20]
5. Create a directory called `/boot_files`. This is where you will store all the necessary files you need to boot the Raspberry Pi. Copy the `u-boot.bin` file into your `/boot_files` directory.

5.4 Raspberry Pi Firmware Setup

Use WSL to clone the Raspberry Pi Firmware repository (see section 5.1).

1. Navigate to the firmware directory that you cloned, and then into the `/boot` directory it contains.
2. Inside the directory there are a number of files. Copy the following files into your `/boot_files` directory:
   - `bcm2711-rpi-4-b.dtb`
   - `fixup4.dat`
   - `fixup4cd.dat`
   - `fixup4db.dat`
   - `fixup4x.dat`
   - `start4.elf`
   - `start4cd.elf`
   - `start4db.elf`
   - `start4x.elf`
   - `rpi4.build`
   - `test-ifs.bin`

20. "Building with GCC," Das U-Boot, accessed April 7, 2026, https://docs.u-boot.org/en/latest/build/gcc.html.

5.5 Create Boot Script and Config File

Next, you are going to want a command that boots the image, as well as a config in order to ensure your boot goes smoothly. In your `/boot_files` directory, follow the following steps:

1. Create a file called `boot.cmd`. This will contain a script that will load your image into memory and boot it.
2. Put the following into your `boot.cmd` file:

```
echo "Trying to load primary image"

fdt addr -c

# Load the fdt first
setenv fdt_addr ${fdtcontroladdr}

# Load the image into memory
if load mmc 0:1 0x00080000 ${image}; then
    echo "Loaded primary image into memory"
    sleep 5
    go 0x00080000 ${fdt_addr}

# If it could not be loaded try to load a backup
else
    echo "Failed to load primary image memory"
    if test "${image}" = "ifs-rpi4.bin"; then

        # Loading the backup image
        echo "Trying to load backup image"
        if load mmc 0:1 0x00080000 ifs-rpi4.bin.bak; then
            echo "Loaded backup image into memory"
            sleep 5
            go 0x00080000 ${fdt_addr}

        # Indicate that no image could be loaded
        else
            echo "Failed to load backup image"
        fi
```

```
    # Specify that the boot image failed
    else
        echo "Boot failed for both primary and backup images"
    fi
fi
```

Here is a quick rundown of what this does:

- The first thing the script does set the device tree address.
- The script then loads your image into memory and checks if it was successful. If it was, the image is booted.
- If no image was found or it was not loaded correctly, the script tries to find and load a backup image into memory. If this is successful, the script boots the backup image.

3. The file now needs to be converted into something Linux readable, then into an executable. Run the command `dos2unix boot.cmd`, and then `mkimage -A arm -T script -C none -n "Boot Script" -d boot.cmd boot.scr`. This will generate a file called `boot.scr`.

4. Create a config.txt file. Insert the following into the file:

```
arm_64bit=1
cmdline=startup.txt
device_tree=bcm2711-rpi-4-b.dtb
enable_uart=1
disable_commandline_tags=1
force_turbo=1
gpu_mem=16
max_framebuffers=2
dtparam=sd_poll_once
kernel=u-boot.bin
```

5.6 Create Raspberry Pi Image

You now need to build the Raspberry Pi image you actually want to run.

1. Open your unzipped BSP directory as a new project in QNX Momentics IDE. In the `/images` directory, find the file `rpi4.build` file and replace it with `rpi4.build` the taken from `/boot_files`.

2. Make sure the startup line is the following: `startup-bcm2711-rpi4 -v -D miniuart -W 2500 -u arg -t "below1G:0x11000000:0x20000000"` (this line should be very close to the top of the file).

3. Under `/usr/share/terminfo/q/qansi` please insert file path to `qnx800/target/qnx/usr/share/terminfo/q/qansi` from your local machine.

4. Under the SD Memory Card section, ensure that the card is mounted as a readable directory as follows:[21]

```
devb-sdmmc-bcm2711 cam pnp mem name=below1G sdio addr=0xfe340000,irq=158 dos exe=al
waitfor /dev/hd0 10
mount -e /dev/hd0
waitfor /dev/hd0t12 10
mount -t dos /dev/hd0t12 /fs/sd
```

5. You will find the scripts to run OTA updates at the bottom of your build file. Here is a brief explanation of how the ML-DSA script works:

   - Takes in the new boot image and the ML-DSA signature file
   - Checks that the image, signature, current image, and embedded ML-DSA public key all exist.
   - Starts a background sampler using top so CPU and RAM usage can be recorded during the update.
   - Runs /proc/boot/validate_mldsa using the embedded public key and the provided signature.
   - If validation fails: the update is aborted, no install happens, a failure summary is still written to the log.
   - If validation passes it continues with the following update.
   - Creates a backup of the current image as ifs-rpi4.bin.bak
   - Copies the new image into a temporary staged file, `ifs-rpi4.bin.new`.

21. "Driver Commands," QNX, March 26, 2026, accessed April 7, 2026, https://www.qnx.com/developers/docs/BSP8.0/com.qnx.doc.bsp_raspberrypi.bcm2711.rpi4_8.0/topic/driver_commands.html.

- Replaces the active boot image with the staged one, then syncs to flush writes to disk.
- Stops the sampler and parses the recorded top output to calculate:total OTA time, average CPU. maximum CPU, average RAM, maximum RAM
- Writes those results to ota_summary.log and raw sample data to ota_samples.csv.

6. Finally retrieve the executable files from `/aarch64_bin` and place them within the same directory `/install/archlle/bin`. Then retrieve the `rsapubkey.pem` and `mldsa_pubkey.bin` from the `signatures` file and similarly put into the directory `/install/archlle/bin`.

7. Now that you are done with `rpi4.build`, build the project. This should generate a `ifs-rpi4.bin` image file. Copy this new file into your `/boot_files` directory.

5.7 Sign Raspberry Pi Image

You now need to create the signature for your bin file

1. To create the signature for your bin use the provided files `scr`: `sign_mldsa`, `keys`: `mldsa_prvkey.bin`, and `Makefile`

2. using the command make build the executable file for `sign_mldsa.c`

3. Run the following command `./sign_mldsa mldsa_prvkey.bin ifs-rpi4.bin new-ifs.bin.sig`. This will generate your signed bin file

5.8 Partition the SD Card

If your SD card is not already partitioned and ready for use, you will need to do the following:[22]

1. Connect your SD card via SD card reader to your Windows machine.

2. In Windows PowerShell, run the command `diskpart`. This will open Windows' disk partitioning utility.

3. Run the command `list disk`. You should see a number of selectable disks to choose from. Locate your SD card, and type the command textttselect disk <number of the SD card>.

22. "Prepare a bootable microSD card," QNX, March 26, 2026, accessed April 7, 2026, https://www.qnx.com/developers/docs/BSP8.0/com.qnx.doc.bsp_raspberrypi.bcm2712.rpi5_8.0/topic/common/prepare_sdcard.html?hl=diskpart.

4. Run the `clean` command to wipe the SD card. Make sure you chose the right disk; if not, you may end up wiping data on another unintended disk, including your own Windows machine.

5. To create the new partition, run `create partition primary size=256 offset=1024`.

6. Now, you will want to locate your partition. Run `list partition`, and locate your new partition (since you just wiped the card, it should be partition 1). Select the partition with the command `select partition <number of partition>`.

7. Run the command `active` to mark the partition as active in the Master Boot Record (MBR).

8. You will need to format the file system of the partition to a format that the Raspberry Pi can use to boot from. Run the command `format fs=fat32 quick` to format the partition properly.

9. To leave diskpart, simply run `exit`. Your SD card should now be ready to set up with your boot files.

5.9 Boot the Raspberry Pi

Your setup should be mostly complete. Now, you need to boot the Raspberry Pi with your QNX image.

1. Connect your SD card to your Windows machine. Copy everything from your `/boot_files` directory into your SD card. Replace `ifs-rpi4.bin` with your `ifs-rpi4.bin` file and a copy called `new-ifs.bin`. Also add your new-`ifs.bin.sig` file you created above. Insert your SD card into the Raspberry Pi.

2. Connect the USB to TTL cable to your windows device, and the following TTL ends to the Raspberry Pi's UART pins:

   - The black cable to pin 6
   - The green cable to pin 8
   - The white cable to pin 10
   - DO NOT PLUG IN THE RED CABLE

3. Open PuTTY, and choose the serial connection type. Check your Windows machine's Device Manager to see what Port is connected (should be similar to COM3). Fill the serial line with the port connection you found, and set the speed to 115200. Press Open, and you should see a blank terminal appear.

4. Connect the Raspberry Pi to power. The first time you boot, press any key to interrupt the boot, as we want to access the U-Boot terminal. From here, set an environment variable for the image and boot script, and run the script with the following commands:

   ```
   setenv bootcmd 'setenv image ifs-rpi4.bin; fatload mmc 0:1 ${loadaddr}
   boot.scr; source ${loadaddr}' (THIS IS ALL ONE LINE)

   saveenv

   reset
   ```

5. From here, you should simply be able to watch your image boot. On future boots, you will not need to enter the U-Boot terminal again unless you want to change more environment variables.

5.10 Run the OTA Update

Finally, with your image up and running, we can try to run an OTA update.

1. A network connection will have to be established first. Plug your Raspberry Pi into your Windows machine with an Ethernet cable.
2. On the QNX image, set a static IP with the command `ifconfig genet0 192.168.1.10 netmask 255.255.255.0 up`. Check to see if the network is active with the command `ifconfig genet0`.
3. On your Windows machine, open your windows settings. Navigate to Network & internet, then find the Ethernet connection and click Edit next to IP Assignment, and choose the Manual option. Set the IP Address to 192.168.1.11, the Subnet Mask to 255.255.255.0, and then press save to apply changes.
4. The two devices should now be able to connect to each other. To test this, use the ping command to see if packets are received from either device. On both the Windows Command Prompt and your QNX OS image, the command syntax should be `ping <OTHER_DEVICE_STATIC_IP>`. Both should result in packets being received consistently.
5. Now to run your ML-DSA validated OTA update enter this command in terminal `ksh /proc/boot/ota_update_ifs_rsa.sh /fs/boot/new-ifs.bin /fs/boot/new-ifs.bin.sig`

6 Results and Analysis

6.1 Summary of Findings

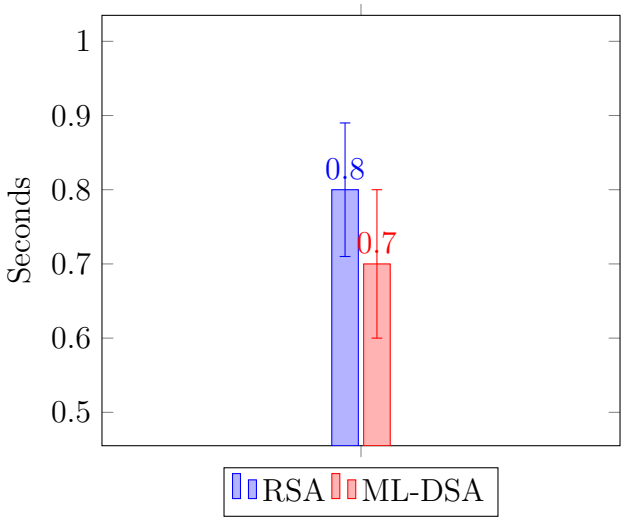
Table 1: Summary of OTA Results

Algorithm	Verify Time (s)	Total OTA (s)	Avg CPU (%)	Max CPU (%)	Avg RAM (MB)	Max RAM (MB)	Sig Size (bytes)
Unsigned	N/A	6.95±0.17	2.73±0.10	3.97±0.12	201±0	201±0	0
RSA	0.8±0.09	7.45±0.15	3.54±0.05	5.27±0.03	199.93±0.03	200±0	256
ML-DSA	0.7±0.10	7.25±0.10	3.53±0.05	5.33±0.01	200.89±0.05	201±0	2420

6.2 Signature Verification Time

Surprisingly, the tests showed a slight reduction in signature verification time between ML-DSA and RSA. This is possibly due to the time taken by OpenSSL's polymorphic key initialization functions to determine which algorithm was used to generate the public key. Overall, however, this did not result in a significant difference in verification time between the two algorithms.

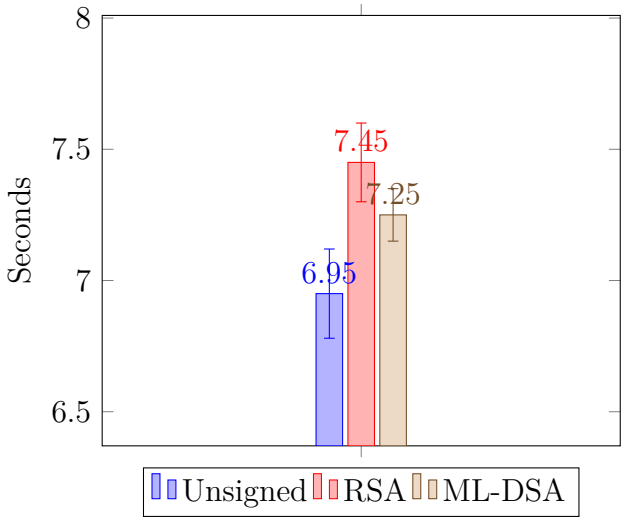
Figure 2: Signature Verification Time by Algorithm



6.3 Total OTA Time

The difference in total OTA time was a similar story to section 6.2. ML-DSA surprisingly had a reduction of 0.2 seconds when compared to RSA and was only 0.3 seconds longer than the unsigned option. Again, however, this is not a terribly significant difference.

Figure 3: Total OTA Time by Algorithm



6.4 Resource Usage

CPU usage across the two algorithms did not differ greatly. There was a <= 0.06 percent difference between both their average and max CPU usages. When compared to the unsigned option however this difference jumps to >= 0.8 and >= 1.3 percent for average and max CPU usage respectively. As for RAM usage, all three options remained around 201MB with RSA having a very minor reduction of 1MB.

Figure 4: CPU Usage by Algorithm

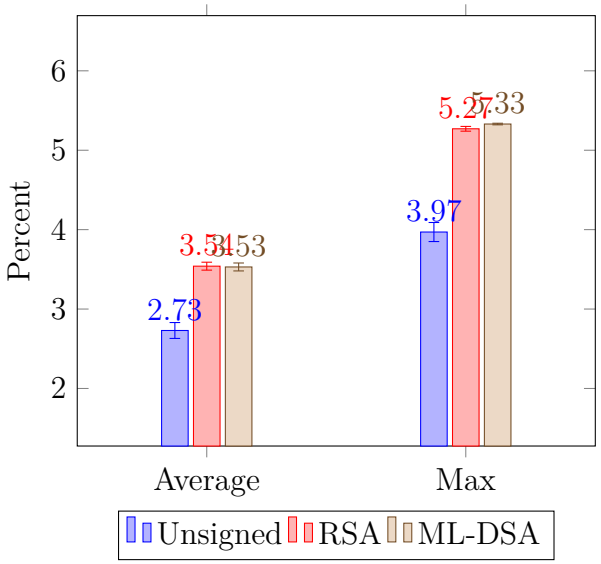
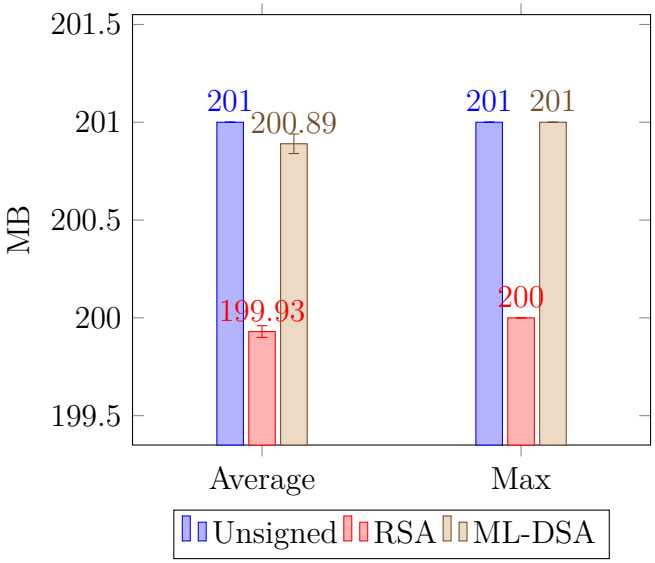


Figure 5: RAM Usage by Algorithm



7 Discussion of Findings

7.1 Practical Meaning of the Results

As seen in section 6, the impact of post-quantum computing signature algorithms on QNX is minimal. Furthermore, when comparing post-quantum solutions such as liboqs to the current OpenSSL standard, these algorithms are actually slightly beneficial to both resource usage and runtime. The biggest drawback comes from the larger key and signature lengths as the added 1056 and 2164 bytes respectively may prove an issue for systems with limited storage. Additionally, for machines which currently choose to forgo implementation of signature verification whatsoever the impacts are much more significant, meaning issues could arise in areas such as CPU usage. This comes mainly in the case of machines making due with the bare minimum spec wise. For most machines, however, it can be concluded that the migration of QNX OS to a post-quantum world is entirely feasible.

7.2 Limitations

A key limitation of this project is that OTA update systems are difficult to evaluate for non specific purposes. Full production deployment depends on more hardware variation, longer testing, and specific device capacity and acceptable resource usage considerations. As a result this work is mainly useful as a controlled experiment to demonstrate theoretical feasibility, though commercial integration of PQC algorithms will likely need to be implemented with specific platforms in mind. This project's evaluation is limited to a single Rasberry Pi 4 running QNX 8.0, which helps compare RSA and ML-DSA in a realistic embedded environment, but doesn't capture how the same design would behave across different processors, storage devices, networks or more limited embedded devices.

Our project ended with an OTA path that is still a simplified version rather than a full update system. In the current code, QNX mounts a single boot partition at /fs/boot and the update script back if the current ifs-rpi4.bin, stage a new image and then replace the active image file directly, so the design is an image and a backup copy. True A/B systems have independent bootable partitions. The fallback logic is also limited because the U-Boot script only tries the backup image if the primary image fails to load, and doesn't track if the system booted successfully, completed initialization or stayed stable after startup, so recovery from runtime failures is currently beyond the code's scope. Several technical limitations prevented us from implementing a full A/B update system. Firstly, the software needs to reliably track which partition is currently being booted so that the correct image can be updated and rolled back. Second, the systems needs a mechanism to detect unsuccessful boots and automatically revert to the previous known good image. Third, the bootloader

is on a separate partition so it needs to manage images stored across 2 different bootable partitions, which adds complexity. Given the time constraints of this assignment, as well as the limited hardware provided by the Raspberry Pi 4 we were not able to achieve this level of complexity.

Lastly, as of OpenSSL 3.5, the library's polymorphic EVP_PKEY functions used for all aspects of cryptography support a variety of post-quantum cryptographic algorithms such as SLH-DSA, ML-KEM, and ML-DSA. Unfortunately, however, QNX currently ships with OpenSSL 3.0 meaning that, for the purposes of this paper, we had to use liboqs in its place. This is significant because post-quantum migration will most likely come in the form of QNX updating to this new version of OpenSSL rather than the widespread adoption of liboqs, which is both external and would require large changes to legacy code. Thus, the results of our tests reflect a future different to what is probable.

7.3 Future Work

Our implementation of A/B partitioning, although having similar behaviour and resiliency, is not pure A/B partitioning. In order to implement this, we would have to separate the disk into at least three partitions: one for the bootloader (in our case U-Boot), partition A for one image, and partition B for another. Although this was attempted, even with the help of an experienced embedded hardware technical support worker at QNX, we were unable to get this functionality working in the allotted time. A potential future improvement would be to spend more time enabling A/B partitioning rather than implementing its behaviour.

Another future improvement to be made is automation as it relates to the networking of the Raspberry Pi. Currently, in order to facilitate OTA transfer, the device must be manually connected to a host device via Ethernet cable and static IP addresses. In addition, the host is running a local server to send the new image, since the images and the Raspberry Pi are not currently configured with internet connectivity. As such, an improvement we could make is to enable and automate internet connectivity. Then, we could host a static server with the files to be sent. This way, the OTA script could be run to pull the new image from the publicly hosted server, or even be automated to poll the server and see if there are new files to pull itself.

There is also other future work to be implemented outside of our immediate control. As mentioned previously in section 7.2, future adoption of post-quantum computing algorithms will most likely come from QNX updating to use OpenSSL 3.5. When this becomes a reality, it is thus imperative to recreate these tests using OpenSSL's built in functionality as opposed to the external liboqs library for accurate results.

8 Conclusion

With great strides being made in the field, quantum computing has become a more prevalent and looming threat than ever before. The birth of a true quantum computer will be the dawn of a new era. It will also cause immeasurable damage to systems all across the globe as quantum-specific shortcuts allow these computers to easily crack legacy cryptographic algorithms. To avoid this, the adoption of post-quantum computing algorithms has become almost essential, with counteractive measures and algorithms such as ML-DSA, ML-KEM and SLH-DSA already being outlined in the NIST standards as of 2024.

From this paper, we learned that the migration of QNX OTA update mechanisms to such algorithms is theoretically feasible. The results of our tests in section 6 show that liboqs' ML-DSA implementation barely impacted metrics such as runtime, CPU usage, and RAM usage when compared to the widely used OpenSSL's RSA implementation. Although the impacts were more significant for systems that implement no authentication whatsoever, this subject was never a concern for them in the first place. Thus, it can be said that a post-quantum world should pose little threat to QNX OTA update mechanisms. The biggest hurdle comes from a lack of internal support from QNX for post-quantum computing cryptographic algorithms. The library used in this paper, liboqs, is external and would require large changes to legacy code. Thus, widespread adoption remains unlikely. Meanwhile, QNX's built-in support for OpenSSL lags behind at version 3.0, meaning access to the library's ML-DSA implementation in version 3.5 is still a little ways off.

The results of this study simply prove that the adoption of post-quantum computing signature algorithms is theoretically feasible. This paper should not be treated as definitive proof. Numerous aspects limited these tests from accurately depicting the cost of future QNX OTA update mechanisms. In addition to the aforementioned lack of OpenSSL 3.5, the OTA update system we tested on was both simplified and generalized such that it reflects no specific software. Thus, while we were able to compare RSA and ML-DSA in a realistic embedded environment, the results do not capture how they might behave or additional limitations brought about by running across different processors, storage devices, networks, or more limited embedded devices.

With study in the field rapidly advancing and work being done every day on both the cryptographic and computing sides of the spectrum, the future remains unforeseeable. Nonetheless, the results of our study show that an optimistic, safe, and easy transition to a post-quantum computing world is not just a dream, but a possible reality.

References

bhess. "liboqs." GitHub. Accessed April 7, 2026. https://github.com/open-quantum-safe/liboqs.

"Building with GCC." Das U-Boot. Accessed April 7, 2026. https://docs.u-boot.org/en/latest/build/gcc.html.

"Driver Commands." QNX, March 26, 2026. Accessed April 7, 2026. https://www.qnx.com/developers/docs/BSP8.0/com.qnx.doc.bsp_raspberrypi.bcm2711.rpi4_8.0/topic/driver_commands.html.

"EVP_PKEY-RSA." OpenSSL Documentation. Accessed April 7, 2026. https://docs.openssl.org/3.4/man7/EVP_PKEY-RSA/.

Holzmayr, Josef. "Robust OTA updates with A/B Partitions for Linux devices." Mender, March 26, 2025. Accessed April 7, 2026. https://mender.io/blog/robust-ota-updates-with-partitions-for-linux-devices.

Ivezic, Marin. "Trust Now, Forge Later (TNFL) – The Overlooked Quantum Threat." ISARA, September 10, 2025. Accessed April 7, 2026. https://postquantum.com/post-quantum/trust-now-forge-later/.

"mkqnximage." QNX, August 11, 2025. Accessed April 7, 2026. https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.utilities/topic/m/mkqnximage.html.

"ML-DSA A Quantum Cryptography Solution for Digital Signing." digicert, April 7, 2024. Accessed April 7, 2026. https://www.digicert.com/insights/post-quantum-cryptography/mldsa.

"Module-Lattice-Based Digital Signature Standard." NIST, August 13, 2024. Accessed April 7, 2026. https://csrc.nist.gov/pubs/fips/204/final.

"Module-Lattice-Based Digital Signature Standard." National Institute of Standards and Technology. Accessed April 7, 2026. https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.204.pdf.

"Post-Quantum Cryptography." NIST, December 11, 2025. Accessed April 7, 2026. https://www.nist.gov/pqcrypto.

"Prepare a bootable microSD card." QNX, March 26, 2026. Accessed April 7, 2026. https://www.qnx.com/developers/docs/BSP8.0/com.qnx.doc.bsp_raspberrypi.bcm2712.rpi5_8.0/topic/common/prepare_sdcard.html?hl=diskpart.

"Raspberry Pi computer hardware." Rasberry Pi. Accessed April 7, 2026. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html.

"RSA Algorithm in Cryptography." geeksforgeek, July 23, 2025. Accessed April 7, 2026. https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/.

Singleton, Siara. "How to Test OTA Updates Without Bricking Devices." Memfault, May 15, 2025. Accessed April 7, 2026. https://memfault.com/blog/ota-testing-101-the-ultimate-guide.

———. "OTA for IoT: What It Is, How It Works, and Why It Matters." Memfault, April 1, 2025. Accessed April 7, 2026. https://memfault.com/blog/ota-for-iot/.

Truskovsky, Alexander. "Do You Know About Quantum's Threat to OTA Software Updates?" ISARA, September 17, 2017. Accessed April 7, 2026. https://www.isara.com/blog-posts/quantums-threat-ota-software-updates.html.

"Understanding Shor's And Grover's Algorithms And Their Impact On Cybersecurity." Fortinet, 2025. Accessed April 7, 2026. https://www.fortinet.com/resources/cyberglossary/shors-grovers-algorithms.